

CHAPTER 33

More on Optimization Techniques

In this chapter, we'll go over some other optimization techniques for improving the ability of a neural network to learn complex patterns in a dataset.

Momentum

Momentum is a technique for improving the convergence speed of stochastic gradient descent (SGD) optimization. Remember that stochastic gradient works by learning the direction of steepest descent by evaluating a training example at each time step to optimize the weights of the network. Momentum improves on this by calculating the average of previous gradients in a process called exponentially smoothed averages. It then uses this computed average to continue to move in the direction of steepest descent. By doing so, it quickens the learning process. In computing this exponentially decayed average, a momentum hyper-parameter is introduced to control how the weight parameters are updated. Figure 33-1 shows an example of stochastic gradient descent with and without momentum as it converges in a function space. In TensorFlow 2.0, momentum is added to a SGD optimizer by adjusting the **'momentum'** parameter of the **SGD method**, **'tf.keras.optimizers.SGD(momentum=[float >=0])'**. The momentum value must be a float value that is greater or equal to 0 that accelerates SGD in the relevant direction and dampens oscillations.

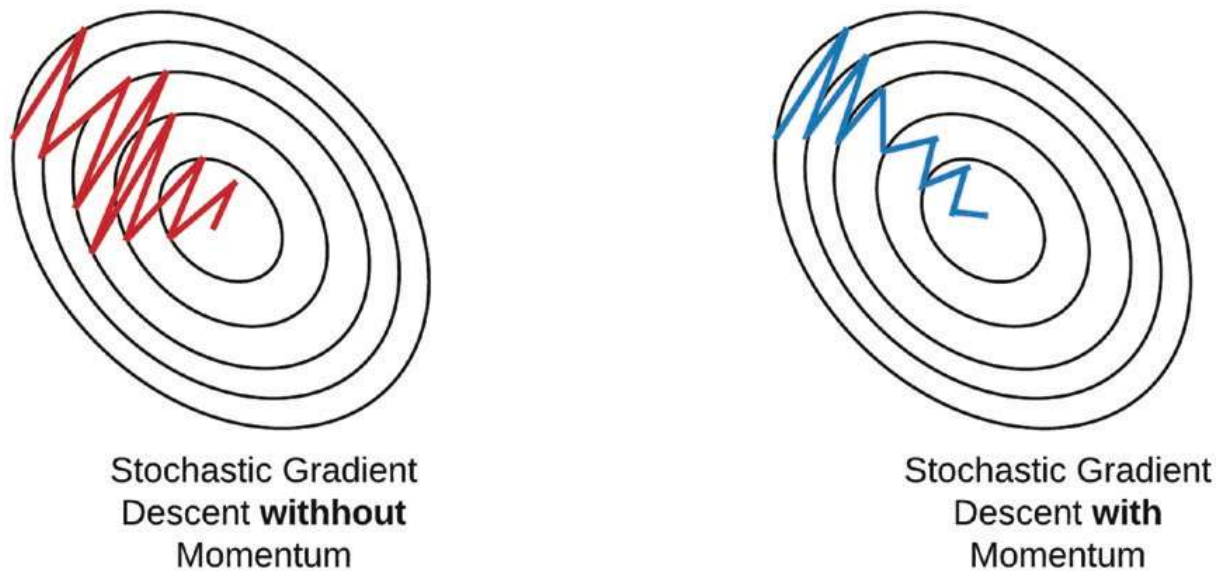


Figure 33-1. SGD with and without momentum

Variable Learning Rates

Remember that the learning rate controls how large a step the gradient descent algorithm makes when moving in the direction of steepest descent. If the learning rate is large, the algorithm takes larger steps in the direction of the steepest gradient, as is faster. However, the algorithm may overshoot the global minimum and fail to converge. But if the learning rate is set to a small number, closer to zero, the algorithm converges slowly, but it is more guaranteed to converge.

Variable learning rates are a set of techniques for adjusting the learning rate of the gradient descent algorithm at every time instance while training. These methods are also called learning rate scheduling. Examples of variable learning rates include

- **Step decay:** This method reduces the learning rate by a constant factor after a certain number of iterations.
- **Exponential decay:** The exponential decay adapts the learning rate following an exponential distribution.
- **Decay proportion:** This method reduces the learning rate by a ratio of 1 over the time instance, t . The learning rate decay can be adjusted by modifying the proportionality constant.

In TensorFlow 2.0, the **'decay'** parameter of the selected optimizer from the **'tf.keras.optimizers'** module allows time inverse decay of learning rate.

Adaptive Learning Rates

Adaptive learning rate, on the other hand, re-adjusts the learning rate in accordance with the training data. It basically uses a different learning rate for each parameter and adapts it during training. These techniques are based on the observation that each parameter results in a different type of gradient. The following list outlines types of adaptive learning rates in use and their method calls in TensorFlow 2.0:

- AdaGrad: **tf.keras.optimizers.Adagrad()**
- AdaDelta: **tf.keras.optimizers.Adadelta()**
- RMSProp: **tf.keras.optimizers.RMSprop()**
- Adaptive Moments, (Adam): **tf.keras.optimizers.Adam()**

However, AdaGrad performs poorly when used for training deep learning models due to its monotonic learning rate which could be too aggressive, and learning may stop early during training. As of now, there is no proven best optimization technique, so the choice of the optimization technique is down to the preference of the model designer.

This chapter surveys some other techniques for optimizing the weights of a deep neural network. These techniques have implementations in deep learning libraries such as Tensorflow and Keras and can be explored as hyper-parameters when designing a neural network solution for a particular learning use case.

In the next chapter, we will discuss techniques for applying regularization to a deep neural network to prevent overfitting.